

Statistical Learning with High-dimensional Data



Pr. Charles BOUVEYRON

Professor of Statistics and Artificial Intelligence
Chair of the Institut 3IA Côte d'Azur
Deputy Scientific Director of the Institut 3IA Côte d'Azur
Head of the Maasai research team
Université Côte d'Azur & Inria

✉ charles.bouveyron@inria.fr - [🐦 @cbouveyron](https://twitter.com/cbouveyron)

Outline

1. Introduction
2. Reminder of descriptive statistics
3. The learning process
4. Supervised learning
5. Unsupervised learning
6. Learning in high-dimensions
7. Dimension reduction
8. Clustering in high dimensions

Unsupervised Learning : a difficult task?

It is indeed a difficult task because we work with less information than in the supervised case, but some tasks are by nature unsupervised and it is not possible to escape this nature:

- image denoising
- autonomous cars.
- visualization
- ...

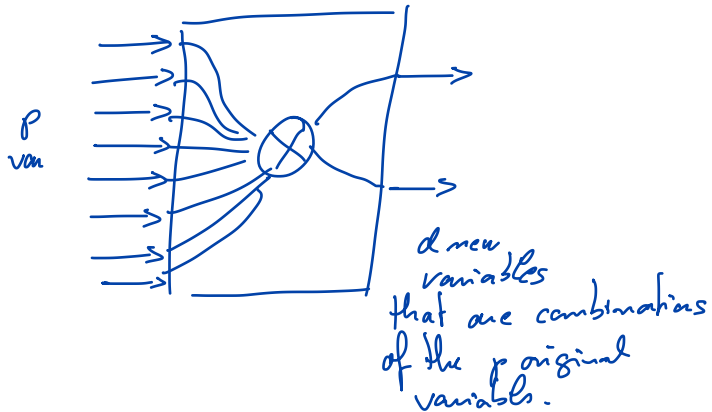
In this course, we will cover dimension reduction and clustering:

- dim. reduction : $X \in \mathcal{X}^p \xrightarrow{\text{learn}} Y \in \mathbb{R}^d$
where $d \ll p$
- clustering : $X \in \mathcal{X}^p \xrightarrow{\text{learn}} Y \in \{1, \dots, K\}$

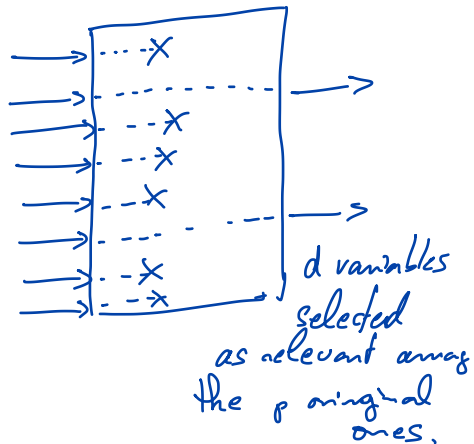
5.1. Dimension reduction:

There are in fact two main families for this task:

(i) feature extraction



(ii) variable selection

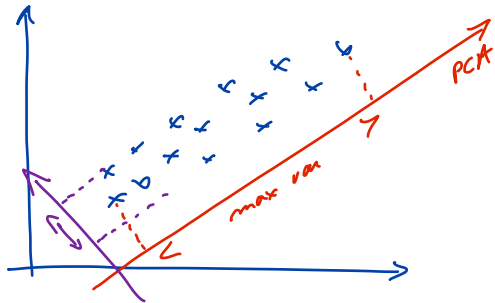


⇒ feature extraction methods (PCA, PLS, ...) are really more efficient in term of information preservation, whereas feature selection methods are better for explainability.

⇒ feature extraction is also far faster than feature selection.

A quick refresher of PCA (and its probabilistic version)

The goal of Principal Component Analysis (PCA) is to build from the p original variables a set of d new variables, by linear combination, such that the projected data have a maximum variance.



PCA is therefore solution of an optimization problem:

$$Y = XU^* \text{ where } U \text{ is a } p \times d \text{ matrix}$$

$$\text{such that } U^* = \arg\max_U \text{Var}(Y)$$

Theorem: The d first axes of U^* are the eigenvectors
of $\Sigma = \overline{X}^t \overline{X}$ associated with the d largest
eigenvalues of Σ (where $\overline{X}$ are the centered data).

In practice:

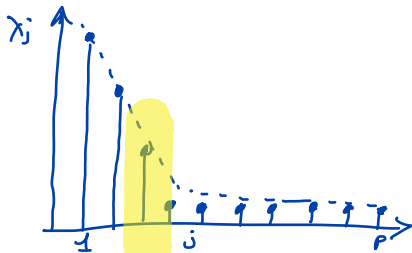
- compute the empirical covariance matrix $\Sigma = \overline{X}^t \overline{X}$
- do the eigendecomposition of Σ
- keep the eigenvectors associated with the largest eigenvalues.

Afterward, we also have to select the appropriate number of dimensions (d):

- the rule of 90% :

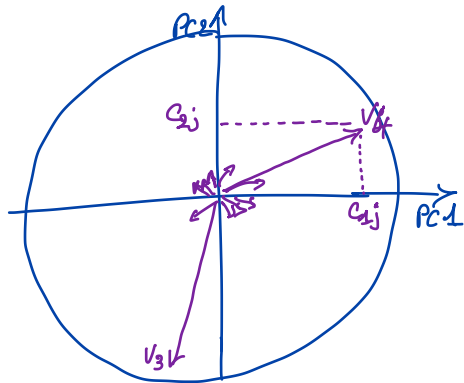
$$d^* = \arg \min_d \frac{\sum_{j=1}^d \lambda_j}{\sum_{j=1}^p \lambda_j} \geq 90\%.$$

- the rule of the elbow in the eigenvalue scree :



$\Rightarrow d^* = 3$ here.

In order to try to understand the relationships between the new PCA axes and the p original variables, it is possible to use the correlation circle:



where c_{ij} is the correlation between the i^{th} axis of PCA and the j^{th} variable

$$c_{ij} = \sqrt{\lambda_j} \cdot u_{ij}^*$$

where PC1 here means the first axis of PCA (i.e. the 1st column of U^*)

A few years ago, Tipping & Bishop (1996) proposed a probabilistic version of PCA: probabilistic PCA (PPCA)

PPCA assumes the following model:

Y: the smaller dimension version of X or we call the latent variables

X: the observed variables

$$X_{\in \mathbb{R}^p} = Y \cdot U^t + \epsilon$$

with

$$\epsilon_{\in \mathbb{R}^p} \sim \mathcal{N}(0, \sigma^2 I_p) \text{ and } Y_{\in \mathbb{R}^d} \sim \mathcal{N}(0, I_d)$$

and U is a $p \times d$ transformation matrix.

Interestingly, the marginal distribution of X is:

$$X \sim \mathcal{N}(0; U^t U + \sigma^2 I_p)$$

It is then possible to use Maximum Likelihood to estimate the parameters of the model: $\theta = \{U, \sigma^2\}$

Here, the problem of PCA has been reformulated as a problem of statistical inference.

Theorem: the d first axes of U are the eigenvectors of $\Sigma = \bar{X}^t \bar{X}$ associated with the d largest eigenvalues.

! Thanks to Tipping & Bishop, we have a justification under a Gaussian assumption of PCA.

$\Rightarrow$ this model also opens the door to many new statistical methods designed for HD data.

In particular:

- it is possible to use **model selection** for choosing the dimension d .

↳ Model selection assumes that PPCA with $d=2$ and PPCA with $d=3$ are 2 two different models and we would like to decide which one is better for the data at hand.

$$\mathcal{M}_1 = \{ \text{PPCA}, d=2 \}$$

$$\mathcal{M}_2 = \{ \text{PPCA}, d=3 \}$$



Model selection criteria are based on the log-likelihood of the model but penalized by the complexity of the model:

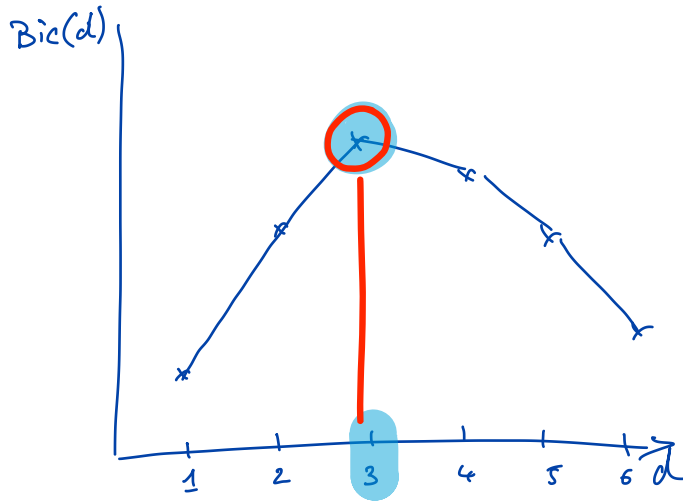
- $AIC(\mathcal{M}) = \log(\mathcal{L}(X; \hat{\theta})) - \underline{\gamma(\mathcal{M})}$

- $BIC(\mathcal{M}) = \log(\mathcal{L}(X; \hat{\theta})) - \frac{\gamma(\mathcal{M})}{2} \log(n)$

where $\gamma(\mathcal{M})$ is the number of free parameters in $\mathcal{M}$.

For instance, in the case of PPCA, $\gamma(\mathcal{M}) = p \times d + 1$
nb of elements in σ^2

In practice, we compute BIC for a range of values of d , and we plot the evolution of BIC:



Sparse PCA (Hastie & Tibshirani, 03) :

when we consider real high-dimensional data ($p > 100$), applying PCA or PPCA could be difficult if there is too much correlation between the original variables. In addition, the correlation circle becomes very difficult to read because of the high number of original variables.

The idea of sparse PCA (SPCA) is to ^{force} force the weights of some original variables to be zero or not in U .

PCA

	PC1	PC2	
$U =$	<u>0.8</u>	<u>0.01</u>	v_1
	0.01	0.02	v_2
	0.02	<u>0.85</u>	v_3
	<u>0.001</u>	0.5	v_4
	<u>0.3</u>	<u>0.001</u>	v_5
	<u>0.6</u>	0.01	v_6



Sparse PCA

	PC1	PC2	
$U =$	<u>0.8</u>	0	
	0	0	v_2
	0	<u>0.85</u>	
	0	<u>0.5</u>	
	0	0	
	<u>0.3</u>	0	
	<u>0.6</u>	0	

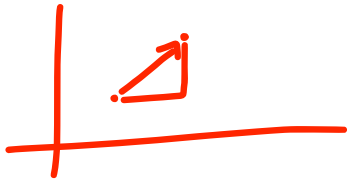
warning: thresholding this matrix is not working because some coefficients maybe small because the variance of the associated variable is very large. the value U_{ij} itself is not directly encoding the importance of the variable

For this, SPCA reformulates the problem of PCA as follows:

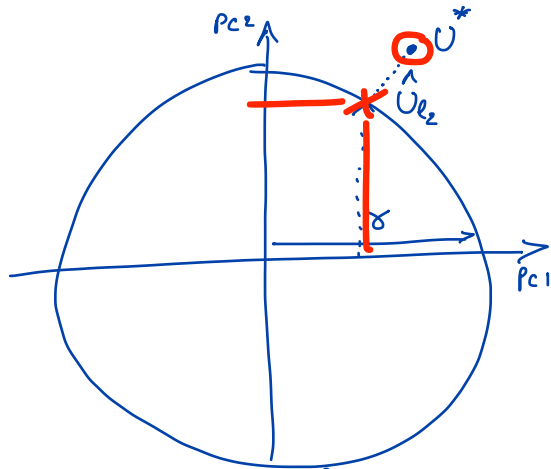
$$U^* = \underset{U}{\operatorname{argmax}} \left\{ \operatorname{Var}(X \cdot U) - \gamma \|U\|_1 \right\}$$

with here the ℓ_1 norm.

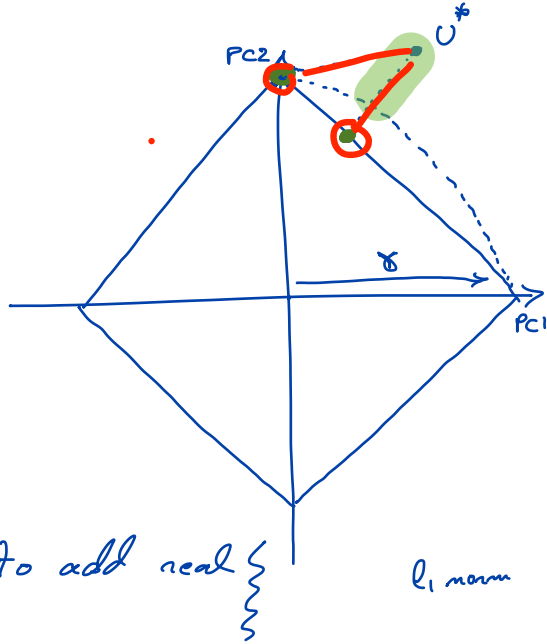
where $\|U\|_1 = \sum_{j=1}^d \sum_{i=1}^p |U_{ij}|$



Why using the ℓ_1 norm forces U to have zeros?



The use of the ℓ_2 norm leads to add real zeros in the matrix U .

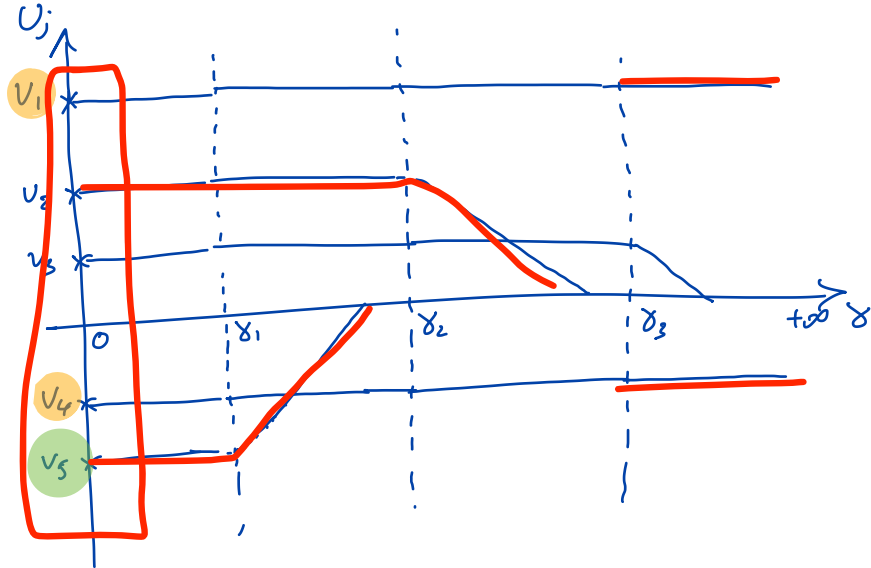


Unfortunately, solving the SPCA problem is possible but complex, and requires the use of a specific optimisation algorithm: the LARS algorithm.

Notice that we also have to choose an appropriate value for the penalty parameter γ :

- if $\gamma = 0$ $\rightarrow$ PCA
- if γ is large $\rightarrow$ very sparse solution.

A final interest of sPCA is the ability of LARS to visualize the weights of the variables according to γ : U_j



Globally sparse PPCA (Bouveyron, Latouche, Raftery, 2016)

Usually, SPCA is quite efficient to simplify the U matrix with zeros but does not succeed to remove irrelevant variables:

$$\begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} \begin{pmatrix} 0.8 & 0 & 0 & 0.2 \\ 0.6 & 0.2 & 0 & 0 \\ 0.1 & 0 & 0 & 0 \\ 0 & 0.2 & 0 & 0 \\ 0 & 0 & 0.6 & 0.8 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

SPCA

.



$$\begin{pmatrix} 0.8 & 0.01 & 0.1 & 0.2 & v_1 \\ 0.6 & 0.2 & 0.01 & 0.01 & v_2 \\ 0 & 0 & 0 & 0 & v_3 \\ 0 & 0 & 0 & 0 & v_4 \\ 0.1 & 0.01 & 0.6 & 0.8 & v_5 \\ 0 & 1 & 0.01 & 0 & v_6 \end{pmatrix}$$

gsPPCA

The goal of gsPPCA is to help identify the relevant original variables for describing the data with PPCA.

The model of gsPPCA:

epsilon1: noise of irrelevant variables

epsilon2: noise of relevant variables

$$X = \underbrace{V W}_{\text{relevant}} Y + \bar{V} \epsilon_1 + V \epsilon_2$$

where: $V = \text{diag}(\underline{v})$ and $\underline{v}_i \in \{0, 1\}$, $\forall i = 1 \dots p$.

$$\underline{\epsilon_1} \sim \mathcal{N}(0, \sigma_1^2 \mathbf{I}_p), \quad \underline{\epsilon_2} \sim \mathcal{N}(0, \sigma_2^2 \mathbf{I}_p)$$

$$\underline{w_{ij}} \sim \mathcal{N}(0, \alpha^{-2}) \quad \underline{Y} \sim \mathcal{N}(0, \mathbf{I}_d)$$

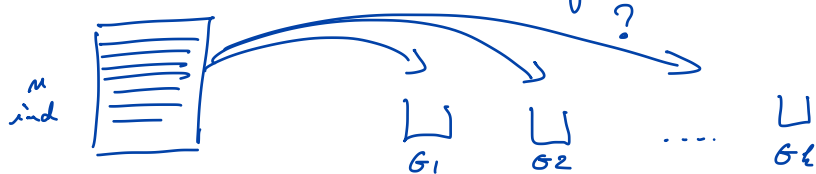
We proposed a specific inference algorithm to estimate the different parameters:

- v which indicate which variables are relevant.
- w_{ij} is the weights of the relevant variables.

Clustering :

Clustering is based on a single idea : assigning individuals to groups such that people in the same group behave similarly and people in different groups behave differently.

This problem is nevertheless a complex one due to the combinatorial nature of the task.



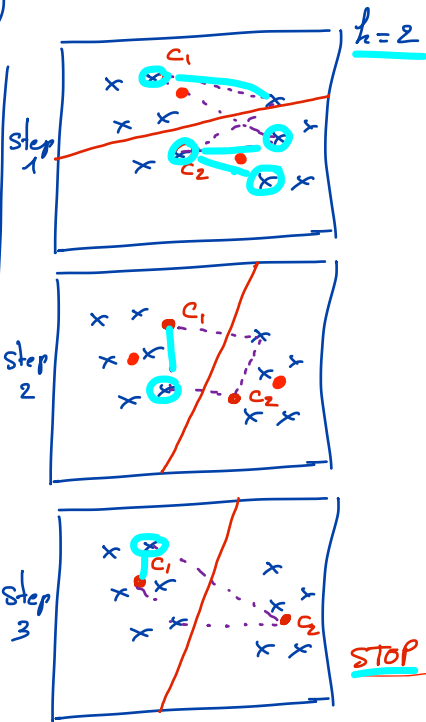
To approximate the best clustering partition, we have a certain number of algorithms to cluster data in a reasonable time:

- k -means.
- hierarchical clustering.
- EM algorithm for Gaussian mixtures.
- spectral clustering
- ...

The k -means algorithm (Lloyd, 1956)

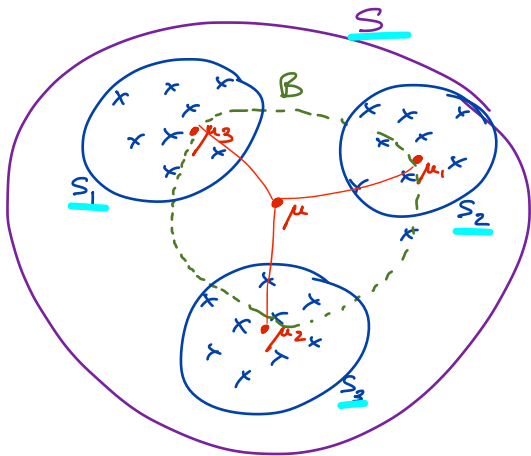
The algo: for a fixed value of k

- initialization: choose k centers.
- loop:
 - (i) compute the distances between all observations and the k centers.
 - (ii) assign each observation to the closest center
 - (iii) recompute the centers.
- stop: when the groups are stable



k-means is a simple but extremely efficient clustering algorithm. Its only limitation is the need to know in advance the number k of groups to find in the data.

It is nevertheless possible to find k automatically if we succeed to define a measure of clustering quality. Such a measure can be defined relying on the notion of variance.



variance of the means

- the between variance:

$$B = \frac{1}{n} \sum_{h=1}^k n_h (\mu_h - \mu)^2$$

It exists three notions of variance:

- the total variance:

$$S = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

- the within variance:

mean of variances

nk: number of individual in group k

$$W = \frac{1}{n} \sum_{h=1}^k n_h S_h$$

where S_h is the variance of group h.

Theorem of Huygens:

$$S = W + B$$

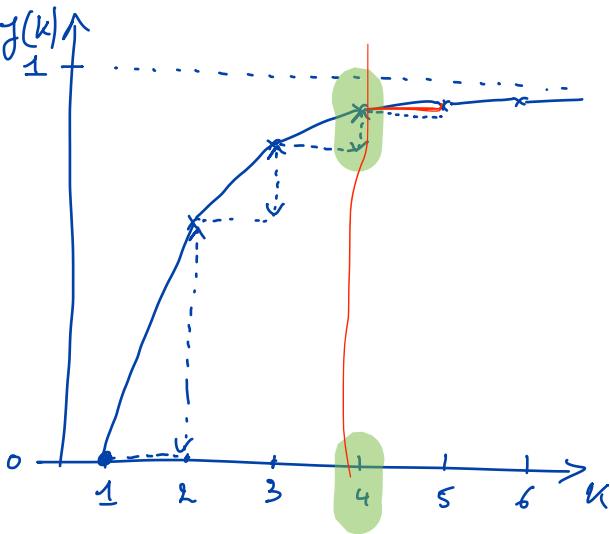
It is therefore possible to define the following criterion :

to maximize it we need to have B large and W small

$$f(k) = \frac{B(k)}{w(k)} \quad \text{which we have to maximize.}$$

In practice, we prefer to use $f(k) = \frac{\dot{B}(k)}{S}$ which is better since S is fixed and not depending on the clustering

So, to choose k in k -means, we can observe the evolution of $J(k)$ according to k .



The best value of k for a specific data set is the value for which adding another group ($k+1$) is not improving significantly the criteria $J(k)$.

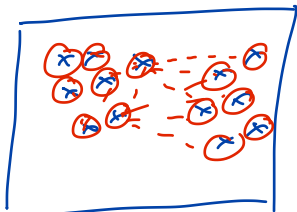
Summary on k-means:

- ⊕ simple and efficient algorithm ($\rightarrow$ reference method!)
- ⊕ the centers can be easily interpreted as the average individual of each group.
- ⊕ in combination with the elbow method on $J(k)$, we can easily select the best k .
- ⊖ k-means is sensitive to initialization and to outliers.
- ⊖ it has the tendency to form groups of the same size.

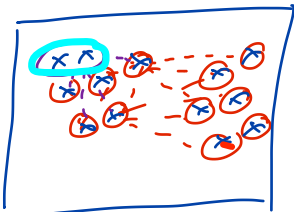
Hierarchical clustering:

The algo:

- initialization: each individual is assigned to its own group. ($K=n$)
- loop: (i) compute the distance between all groups.
(ii) gather the two groups that are the closest.
- stop: when it remains only 1 group.



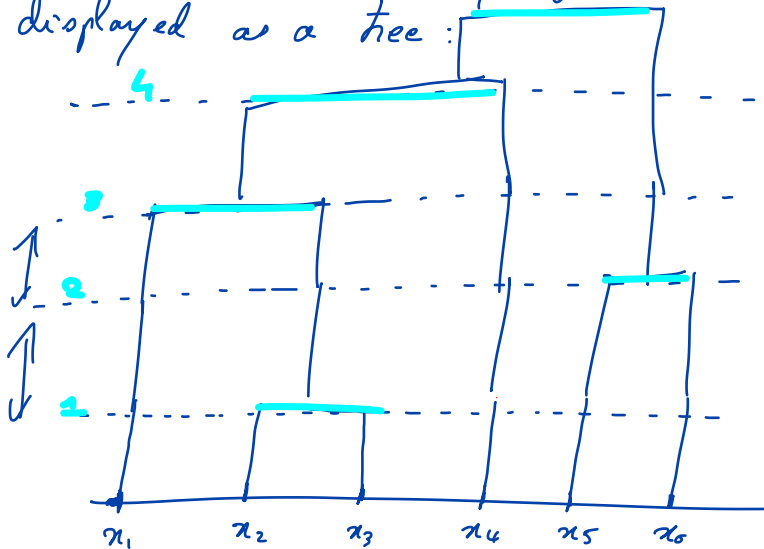
Step 1



Step 2



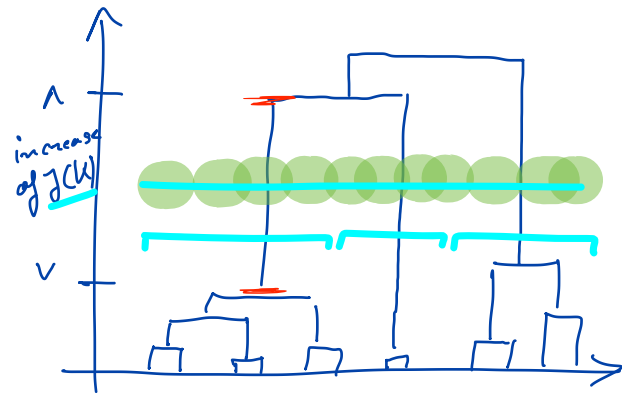
HC is in particular very appreciated due to the fact it produces a hierarchy of clustering, which can be displayed as a tree:



Rank: this plot is called a dendrogram.

In order to choose K , it is again possible to rely on the elbow technique on the criterion $J(K)$.

Interestingly, it has been proposed to incorporate the visualization of $J(K)$ within the dendrogram;



we cut the tree at the largest gap!

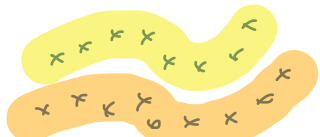
HC is in fact a family of algorithms, because we can play with the notion of distance between the groups:

- the complete linkage:

$$d(A, B) = \max \{ d(a, b) / \begin{matrix} a \in A \\ b \in B \end{matrix} \}$$

- the single linkage:

$$d(A, B) = \min \{ d(a, b) / \begin{matrix} a \in A \\ b \in B \end{matrix} \}$$



- the distance of the centroids:

$$d(A, B) = d(\bar{a}, \bar{b})$$

- the Ward distance:

$$d(A, B) = d(\bar{a}, \bar{b})$$

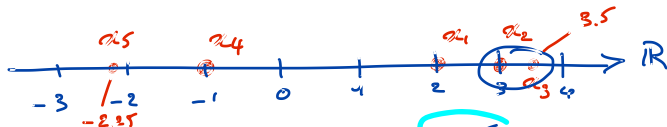
$\frac{1}{n_A} + \frac{1}{n_B}$

it helps producing groups of different sizes.

Summary on HC :

- ⊕ the dendrogram is a very appreciated tool for visualizing the clustering hierarchy.
- ⊕ a flexible algorithm thanks to the choice of the distance between the groups
- ⊕ we have the ability to choose K .
- ⊖ HC has a very high complexity ($O(n^2 \lg n)$)
↳ it is not possible to use it for $n \geq 5000$.

Exercise: Cluster the following data with **HC** associated with the **complete linkage**



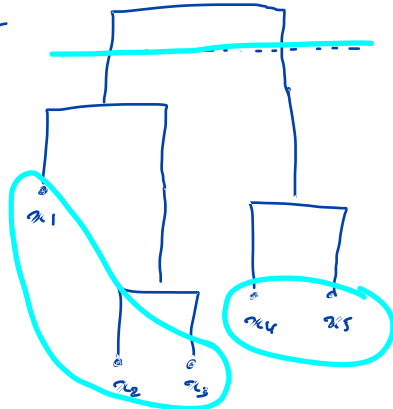
	x_2	x_3	x_4	x_5
x_1	1	1.5	3	4.25
x_2		0.5	4	5.25
x_3			4.5	5.75
x_4				1.25

A shaded triangle is drawn in the lower-left corner of the table, indicating the distance between x_2 and x_3 is 0.5.

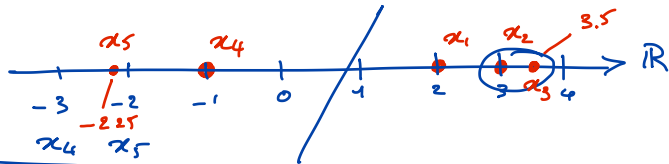
	x_2	x_5
x_1	1.5	4.25
x_3		5.75

$$\max \{ d(x_1, x_2), d(x_1, x_3) \} = \max \{ 1, 1.5 \} = 1.5$$

	x_3	x_4	x_5
x_1	1.5	3	4.25
x_3		4.5	5.75
x_4			1.25



Exercise: Cluster the above data with k -means with $K=2$ and x_1 and x_2 as initial centers.



	x_1	x_2	x_3	x_4	x_5
$C_1 = x_1$	0	1	1.5	3	4.25
$C_2 = x_2$	1	0	0.5	4	5.25

$$\hookrightarrow C_1 = \text{mean}(x_1, x_4, x_5) \approx -0.4$$

$$C_2 = \text{mean}(x_2, x_3) = 3.25$$

	x_1	x_2	x_3	x_4	x_5
C_1	2.4	3.4	3.3	0.6	1.65
C_2	1.25	0.25	0.25	4.25	5.5

$$\hookrightarrow C_1 = \text{mean}(x_4, x_5) \quad , \quad C_2 = \text{mean}(x_1, x_2, x_3)$$